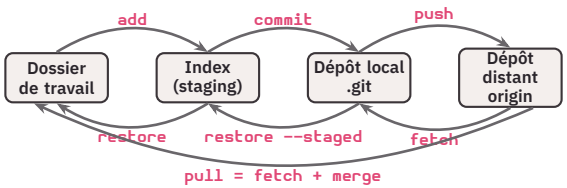
Git | recto · les commandes

`git commande --option ref # commentaire «à remplir»`

Les 4 zones

1 main



`diff`	dossier/index	pas encore indexé
`diff --staged`	index/HEAD	ce que prendra le commit
`diff HEAD`	dossier/HEAD	tout ce qui a changé

HEAD = le commit sur lequel on est (en général le bout de la branche courante).

Configurer et démarrer

```
git config --global user.name "«Prénom Nom»"
git config --global user.email mail
git config --global init.defaultBranch main
git config --global core.editor "code --wait"
git config --list --show-origin  # qui définit quoi
git init                         # nouveau dépôt ici
git clone url dossier            # copie + remote origin
```

.gitignore (à la racine, commité)

```
build/     # un dossier
*.log      # un motif
!keep.log  # une exception
```

Fichier déjà suivi : `git rm --cached f` puis commit, sinon il reste suivi.

Enregistrer

```
git status -sb                  # état court + branche
git add fichier                 # dossier -> index
git add -p                      # morceau par morceau
git add -A                      # tout : ajouts, suppr.
git commit -m "«message»"       # index -> commit
git commit -am "«message»"      # add des suivis + commit
git commit --amend --no-edit    # compléter le dernier
git restore fichier             # jeter les modifs
git restore --staged fichier    # désindexer
git rm fichier                  # supprimer (suivi)
git mv ancien nouveau           # renommer (suivi)
```

Un commit = une idée, avec un message court à l'impératif, ex. `"Fix FIFO full flag"`.

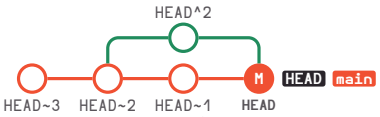
Mettre de côté (stash)

```
git stash                       # range tout, revient à HEAD
git stash push -u -m "«msg»"    # + non suivis, nommé
git stash list                  # stash@{0}, stash@{1}...
git stash pop                   # réapplique + supprime
git stash apply stash@{1}       # réapplique, garde
git stash drop stash@{0}        # supprime
```

Pour changer de branche avec du travail en cours, ou faire un `pull` propre.

Désigner un commit

2 feat/branches



`HEAD~2`	2 commits en arrière (1er parent)
`HEAD^2`	2e parent d'un commit de fusion
`a1b2c3d`	hash abrégé (4 caractères min.)
`origin/main`	copie locale de la branche distante
`HEAD@{2}`	où était HEAD il y a 2 mouvements
`main..feat`	commits de feat absents de main

Historique

```
git log --oneline --graph --all  # l'arbre complet
git log -p fichier               # avec les diffs
git log --author=nom --since=2.weeks
git log main..feat               # ce que feat ajoute
git show hash                    # détail d'un commit
git diff a b -- fichier          # entre deux commits
git blame fichier                # auteur de chaque ligne
```

Trouver le commit fautif (bisect)

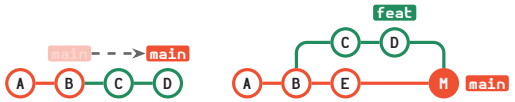
```
git bisect start
git bisect bad         # la version actuelle bugue
git bisect good v1.0   # celle-ci marchait
# git extrait le milieu : tester, puis good / bad
git bisect reset       # fin : retour à HEAD
```

Branches

```
git branch                      # liste (* = courante)
git branch -a -vv               # + distantes et suivi
git switch b                    # aller sur «b»
git switch -c b départ          # créer + y aller
git checkout -b b               # idem (ancienne forme)
git branch -m ancien nouveau    # renommer
git branch -d b                 # suppr. si fusionnée
git branch -D b                 # forcer : travail perdu
git push -u origin b            # publier + suivre
git push origin --delete b      # suppr. sur origin
```

Renommer sur origin : `--delete`, puis `push -u` la nouvelle.

Fusionner (merge)



fast-forward : feat avance juste

3 voies : commit de fusion M à 2 parents

```
git switch main          # 1. aller sur la cible
git merge feat           # 2. y fusionner feat
git push origin main     # 3. publier
git merge --no-ff feat   # M même si FF : trace
git merge --ff-only feat # refuser si pas FF
git merge --squash feat  # 1 commit, sans lien
git merge --abort        # annuler (conflit)
```

Résoudre un conflit

3 origin/main

```
<<<<<<< HEAD                  # notre version
assign y = a & b;
=======                       # séparation
assign y = a | b;
>>>>>>> feat                  # leur version (arrive)
git add fichier               # après édition : résolu
git merge --continue          # ou rebase --continue
git checkout --ours fichier   # garder notre version
git checkout --theirs fichier # garder la leur
```

⚠ **Piège :** dans un rebase, `--ours` = la base (main) et `--theirs` = vos commits : l'inverse du merge.

Dépôts distants




```
git remote -v                 # liste des distants
git remote add origin url     # lier un dépôt
git fetch                     # télécharger, sans fusion
git pull                      # fetch + merge
git pull --rebase             # fetch + rebase : linéaire
git push                      # envoyer la branche suivie
git push --force-with-lease   # après une réécriture
git branch -u origin/b        # suivre une branche
git remote prune origin       # oublier les suppr.
```

`status` compare à `origin/main`, mis à jour seulement par `fetch`, `pull` ou un `push` réussi. Conseillé : `git config --global pull.rebase true`.

⚠ **Piège :** `--force` écrase le travail poussé par les autres, alors que `--force-with-lease` refuse si origin a bougé depuis votre fetch.

Branches en parallèle (worktree)

```
git worktree add ../projet-feat1 feat1  # sur feat1
git worktree add ../projet-feat2 feat2  # autre, sur feat2
git worktree add -b b ../dossier        # + nouv. branche
git worktree list                       # les dossiers
git worktree remove ../projet-feat1     # retirer
```

Un seul `.git` partagé, un dossier par branche : synthétiser feat1 et feat2 en même temps et comparer, sans `switch`. Sans branche, `worktree add ../x` crée la branche `x`.

⚠ **Piège :** une branche ne peut être extraite que dans un seul worktree à la fois.

Travailler en équipe

```
git switch -c feat/x main       # 1. une branche
git fetch                       # 2. se mettre à jour
git rebase origin/main          #    avant de pousser
git push -u origin feat/x       # 3. publier
# 4. merge request : relecture, CI, fusion
git switch main && git pull     # 5. récupérer
git branch -d feat/x            # 6. ménage local
```

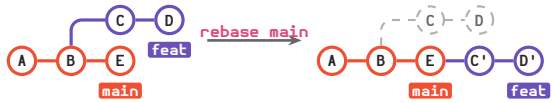
Petites branches, fusionnées vite : moins de conflits. Jamais de commit direct sur `main` partagée.

Git — verso · réécrire, annuler, organiser

`git commande --option ref # commentaire «à remplir»`

Rebaser une branche

4 `feat/rebase`



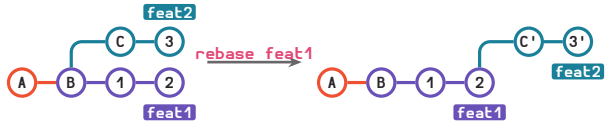
C, D rejoués après E : copies C', D' (nouveaux hash), historique linéaire.

```
git switch feat
git rebase main                # rejoue feat sur main
git rebase --continue          # conflit corrigé + add
git rebase --skip              # sauter ce commit
git rebase --abort             # tout annuler
git push --force-with-lease    # feat déjà poussée
```

Pendant un conflit HEAD est détachée : normal, `--continue` rattache la branche à la fin. Branche restée en arrière : `git switch -C feat` la force sur HEAD (= `checkout -B`).

⚠**Piège :** après le rebase, pas de `git pull` : il refusionnerait les anciens C, D (doublons). Pousser avec `--force-with-lease`.

Branches empilées



feat2 partait de feat1 avant fix 1 et 2 : on la recale.

```
git switch feat2
git rebase feat1                    # repart de feat1
# au choix, à la place :
git rebase --onto main feat1 feat2  # feat2 seule sur main
git rebase --update-refs main       # tout, feat1 suit
```

`--update-refs` 2.38 déplace toutes les branches posées sur les commits rejoués.

Rebase interactif : nettoyer

```
git rebase -i HEAD~3  # les 3 derniers commits
```

■ **l'éditeur : plus ancien en haut, déplacer = réordonner**

```
pick a1b2c3d Add FIFO
squash e4f5a6b Fix FIFO full flag  # fondu au précédent
fixup c7d8e9f typo                 # idem, message jeté
```

`pick`	garder tel quel
`reword`	garder, changer le message
`edit`	s'arrêter pour `commit --amend`, puis `--continue`
`squash`	fondre dans le précédent, messages réunis
`fixup`	idem, garde le message du précédent
`drop`	supprimer le commit (ou effacer la ligne)



■ **plus rapide : marquer, puis ranger tout seul**

```
git commit --fixup hash           # corrige «hash»
git rebase -i --autosquash base   # fixup placés auto
```

reset : reculer la branche

5 `fix/annuler`



`reset HEAD~1` : main recule sur B ; C n'est plus sur aucune branche (reflog)

	branche	index	dossier	modifs de C
`--soft`	✓	–	–	restent indexées
`--mixed`	✓	✓	–	restent, non indexées (défaut)
`--hard`	✓	✓	✓	sont perdues

```
git reset --soft HEAD~1    # défaire le commit
git reset HEAD~1           # idem + désindexer
git reset --hard HEAD~1    # tout effacer
git reset --hard origin/b  # coller au distant
```

Supprimer un commit

■ **pas encore poussé**

dernier	`reset --soft HEAD~1` (garder) ou `--hard` (effacer)
au milieu	`rebase -i HEAD~N`, `drop` sur sa ligne

■ **déjà poussé**

équipe	`git revert hash` puis `push` : un commit inverse, le passé reste intact
perso	`reset --hard HEAD~1` puis `push --force-with-lease`



■ **sur les autres machines, après une réécriture**

```
git fetch
git rebase --abort          # si un rebase traîne
git reset --hard origin/b   # s'aligner sur origin
```

Petites réparations

```
git commit --amend -m "«msg»"  # mauvais message
git add oubli                  # fichier oublié :
git commit --amend --no-edit   # ... dans le dernier
git restore --source=hash f    # un fichier ancien
git revert -m 1 hash-merge     # annuler une fusion
git clean -nd                  # non suivis : lister
git clean -fd                  # ... et supprimer
```

■ **commit fait sur la mauvaise branche**

```
git branch bonne          # garder le commit ici
git reset --hard HEAD~1   # l'ôter de la courante
git switch bonne          # il y est
```

`--amend` réécrit le commit : seulement s'il n'est pas encore poussé.

Filet de sécurité : reflog

```
git reflog                  # HEAD@{0}, HEAD@{1}...
git reset --hard HEAD@{2}   # revenir avant l'erreur
git branch rescue hash      # repêcher un commit
git reset --hard ORIG_HEAD  # annuler reset/rebase
```

Un commit « supprimé » reste au moins 30 jours dans le reflog (local, par défaut ; 90 jours s'il est encore sur une branche) : seul le jamais commité est perdu.

Copier un commit (cherry-pick)

6 `main`




```
git switch main
git cherry-pick hash-fix3   # copie fix 3 sur main
git cherry-pick -x hash     # + « cherry picked from »
git cherry-pick A^..B       # une plage, A inclus
git cherry-pick --continue  # après un conflit
```

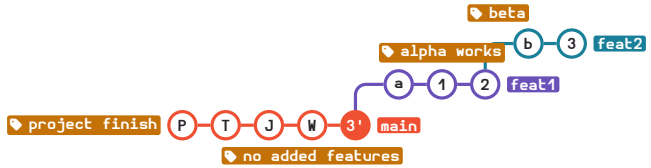
Nouveau commit, nouveau hash ; feat ne change pas. Au prochain `rebase main` de feat, git saute fix 3 (même patch).

Fichiers et dossiers

```
git mv ancien_nom.v nouveau_nom.v  # renommer, suivi gardé
git rm -r junk/                    # suppr. dépôt + disque
git rm -r --cached build/          # ne plus suivre
git commit -m "Remove junk folder"
git switch feat1                   # propager sur feat1 :
git rebase main                    # le renommage suit
git log --follow fichier           # historique complet
```

Renommage reconnu au contenu : au rebase, les modifs de feat1 sur l'ancien nom suivent. Le passé n'est pas réécrit : junk/ reste dans les anciens commits (pour alléger le dépôt : `git filter-repo`, qui réécrit tout).

Structure du projet



P repris · T prise en main · J junk/ supprimé · W adaptation OK · a, b features alpha, beta · 1–3 fix.

```
git switch feat1 && git rebase main       # suit main
git switch feat2 && git rebase feat1      # suit feat1
git push --force-with-lease origin feat1 feat2
git tag -a alpha-works -m "«msg»" feat1   # marquer
git push origin --tags                    # les tags à part
```

Pièges

Réécrire du partagé	rebase, amend, reset sur une branche où d'autres travaillent : ils devront `reset --hard origin/b`.
Perte définitive	`reset --hard`, `restore`, `clean -f` effacent le non commité, sans reflog.
HEAD détachée	les commits faits là ne sont sur aucune branche : `git switch -c b` avant de partir.
Fichiers générés	build/, *.bit, ondes, rapports : dans `.gitignore` dès le début.